Edge Inference Solution Package

Package damage detection for 50 distribution centers · Azure IoT Edge 1.6 LTS · GitOps rollouts

Dennis · Platform Engineering · 11 October 2026 · Version 1.0

This package designs and documents a production edge inference solution that detects package damage at 50 distribution centers, keeps working through uplink outages of up to 72 hours, and ships model and platform updates through automated GitOps rollouts that operations staff can monitor and reverse without raw kubectl. Callout numbers in brackets (for example [C4]) refer to the numbered markers on Figures 1–3; Appendix A maps every callout to its finding.

Decisions at a glance

Decision	Choice	One-line justification
Model runtime	**LiteRT int8** (XNNPACK CPU kernels), 7 MB YOLO detector	Sites are CPU-only 4-core x86 PCs with 2 GB RAM: int8 runs natively on CPU and the small model leaves memory for edgeHub and the offline buffer, whereas the 25 MB fp16 ONNX export would run without fp16 hardware support on these CPUs.
Edge runtime	**Azure IoT Edge 1.6 LTS** — edgeAgent/edgeHub `1.6.0`, host packages held at `1.6.*`	Supported to 14 Nov 2028. IoT Edge 1.5 LTS ends support on 10 Nov 2026, inside peak season, so it is never used as a baseline.
Host OS	**Ubuntu Server 22.04 LTS** (Tier 1, AMD64)	Azure Linux 3.0 is not on the IoT Edge 1.6 supported-platforms list, so it is used only as the container base image.
Offline buffer	edgeHub store-and-forward on the dedicated 8 GB volume; priority routes; 7-day TTL	The 72 h ceiling uses 1.2 GB, 24 % of the 5 GB budget.
Deployment strategy	**Canary** — Flagger (central AKS) + ring-based layered IoT Hub deployments (edge)	A bad release reaches 1 site first; blue-green needs a duplicate stack the 2 GB PCs cannot hold.
GitOps & recovery	Flux v2 AKS extension + GitHub Actions (`az`); dashboard actions and PR Revert for operators	Every change and rollback is a commit; kubectl stays with the platform team.
Diagrams	Embedded as Figures 1–3 and uploaded separately (PDF + numbered PNGs)	Same callout numbers in both.

1. Scope: constraints and assumptions (Checkpoint 1)

The scenario data package fixes the hardware, network, model and volume envelope. Each constraint below drives a concrete design consequence used later in the package.

Area	Constraint (from data package)	Design consequence
Compute	4-core x86_64, 2 GB RAM, no GPU/NPU, 32 GB eMMC	CPU int8 runtime [C3]; hard memory caps per module; no on-site Kubernetes — classic IoT Edge only (`edge.enabled=false` in Helm).
Storage	8 GB dedicated buffer volume, separate from OS/modules	edgeHub queue + frame spool live on it [C5]; Docker log rotation protects eMMC.
Network	~10 Mbps MPLS + LTE failover; outages 45 min avg, 4 h p99, 72 h ceiling, 2–3/site/month	Store-and-forward with priority routes [C4]; drain targets ≤ 15 min (4 h) and ≤ 60 min (72 h) [C19].
Volume	8,000 events/site/day × 50 KB = 400 MB/day; 20 GB/day fleet	IoT Hub S2 × 2 units [C7]; idempotent, batched ingest [C8].
Model	YOLO detector; p95 < 200 ms/frame; mAP@0.5 ≥ 0.88	int8 must pass both gates on reference hardware before Ring 0 [C3].

Area	Constraint (from data package)	Design consequence
Timeline	Peak in 8 weeks; zero-downtime freeze from week 6	Fleet rollout completes by end of week 5 [C28].
People	Operations has limited Kubernetes experience	Operators use the Edge Ops dashboard and GitHub web UI only [C11].
Platform baseline	Pin IoT Edge 1.6 LTS; never 1.5 LTS (EoS 10 Nov 2026)	1.6.0 images, held 1.6.* packages, CI guardrail rejects older runtimes (§3.1).

Assumptions

1. **A1 Operating day.** Conveyors run about 16 h/day, so the peak-hour rate is 500 events/h (25 MB/h).
2. **A2 Trigger.** Each package pass raises one PLC package-present signal (or a camera line-crossing event). Cameras expose ONVIF snapshot URIs, so the edge PC never decodes continuous video [C1].
3. **A3 Goodput.** Sync gets about 60 % of the 10 Mbps link (≈ 6 Mbps) after TLS/AMQP overhead and live traffic. LTE bandwidth is unspecified, so drain targets assume MPLS.
4. **A4 Regions.** Sites are served by 2–3 Azure regions; ACR Premium is geo-replicated to each.
5. **A5 Retention.** Damage evidence and results must survive the 72 h ceiling; devices keep 7 days (TTL), the cloud deduplicates for 14 days, and long-term retention is a business policy outside this scope.
6. **A6 Identity and time.** Each edge PC receives an X.509 identity at imaging (DPS group enrollment) and NTP is reachable on the site LAN; timestamps feed the idempotency key.
7. **A7 Accuracy.** The int8 export meets mAP@0.5 ≥ 0.88; if it does not, the fallback is ONNX Runtime CPU (fp32 or QDQ-int8 export) — see §8.2.
8. **A8 No leaf devices.** Nothing connects to edgeHub from outside the device, so edgeHub publishes no host ports.

Buffer and memory budget

Budget item	Value	Basis
72 h buffer requirement	1.2 GB	400 MB/day × 3 (data package)
edgeHub store budget	5 GB	4× safety margin (data package); runbook alarm at 80 % = 4 GB
Frame spool cap / headroom	1 GB / 2 GB	Remainder of the 8 GB volume: spool for capture→inference, RocksDB compaction headroom
Steady-state ceiling under TTL	≈ 2.8 GB (56 %)	7-day TTL × 400 MB/day — an outage alone cannot reach the 80 % alarm [C21]
Module memory limits	1,184 MiB	edgeAgent 160 · edgeHub 448 · frame-capture 192 · damage-inference 384 (MiB); swap disabled
OS, aziot daemons, moby / free	~450 MiB / ~400 MiB	Limits are ceilings; free memory covers page cache and image pulls

Template inconsistency resolved: the runbook template's “5 GB reserved” is the edgeHub budget inside the data package's 8 GB dedicated volume, not a separate partition.

2. Architecture overview

The solution has three planes (Figure 1). The **edge plane** runs on each site's industrial PC under Azure IoT Edge 1.6 LTS. The **cloud plane** in Azure receives, deduplicates and stores results and hosts the operator dashboard. The **GitOps plane** turns Git commits into rollouts on both. Edge modules are deployed by IoT Hub deployment manifests, never by Helm; the Helm chart deploys only the central orchestrator on AKS.

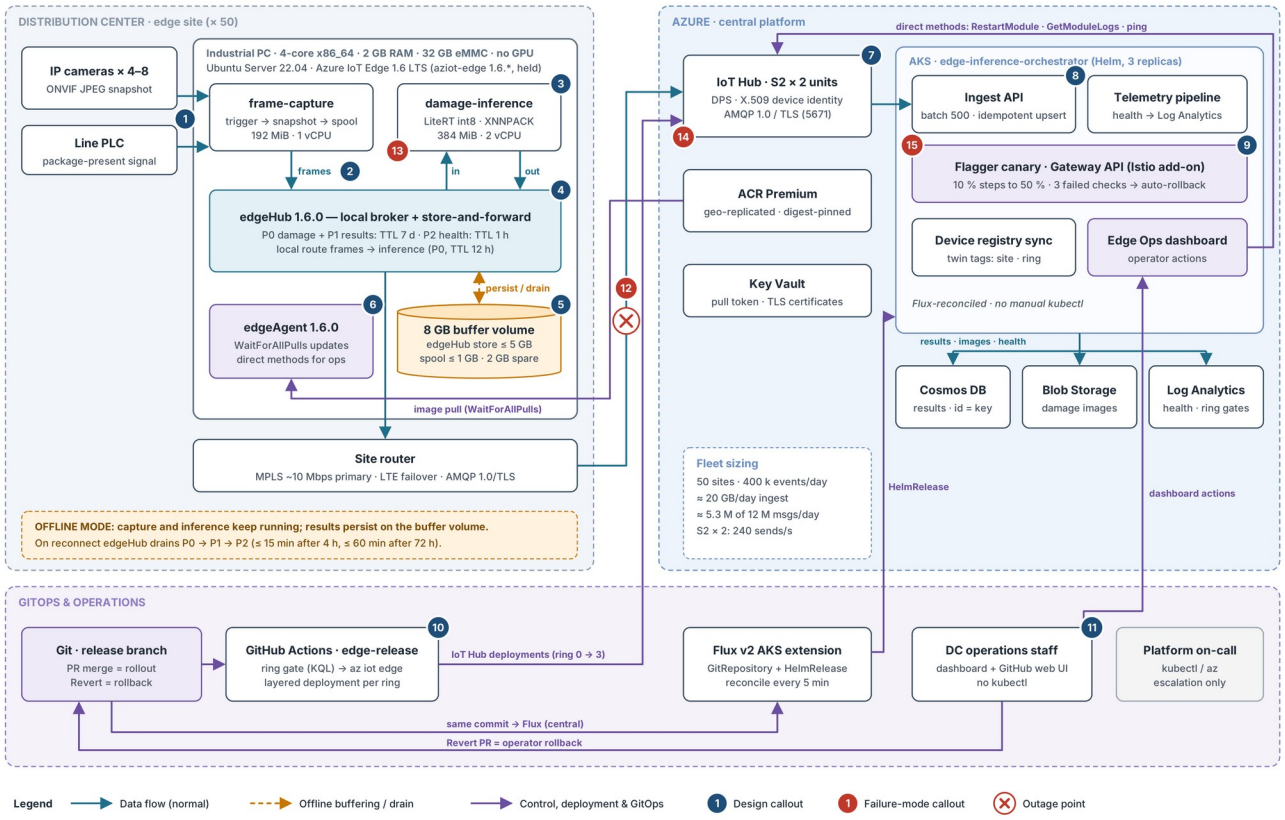


Figure 1. *Each of the 50 sites runs event-triggered capture and LiteRT int8 inference on IoT Edge 1.6 LTS, buffers prioritized results on a dedicated 8 GB volume and syncs over AMQP to IoT Hub, while Git drives both the Flux-managed AKS orchestrator and the ring-based IoT Hub deployments.*

2.1 Component roles

Component	Plane	Role
frame-capture	Edge module	On the package-present signal, pulls one JPEG per package, writes it to the spool and emits a `frames` message carrying the idempotency key [C1][C2].
damage-inference	Edge module	LiteRT int8 detector; emits `damage` (P0), `results` (P1) and `health` (P2) outputs [C3].
edgeHub 1.6.0	Edge runtime	Local broker and durable queue on the buffer volume; upstream AMQP 1.0/TLS to IoT Hub [C4].
edgeAgent 1.6.0	Edge runtime	Applies deployments, pulls all images before restarting anything, serves built-in direct methods [C6].
8 GB buffer volume	Edge host	Separate ext4 filesystem for the edgeHub store and frame spool [C5].
IoT Hub (S2 × 2) + DPS	Cloud	Device identity (X.509), telemetry ingress, deployments, direct methods [C7].
ACR Premium	Cloud	Geo-replicated, digest-pinned images; devices pull with a repository-scoped token from Key Vault.
Orchestrator (Helm on AKS)	Cloud	Ingest API (batch 500, idempotent upsert) [C8], registry sync, telemetry pipeline, Edge Ops dashboard.
Flux v2 + Flagger	GitOps	Reconciles the HelmRelease from Git and canaries every orchestrator change [C9].

Component	Plane	Role
GitHub Actions edge-release	GitOps	Renders the IoT Edge manifest and widens one layered deployment ring by ring [C10].
Cosmos DB · Blob · Log Analytics	Cloud	Results keyed by idempotency key · damage images · health metrics and ring gates.

2.2 Data flow — normal and offline

Normal flow. (1) The PLC signals a package; frame-capture fetches one snapshot over the site LAN [C1]. (2) It writes the JPEG to the spool and sends a small `frames` message on a P0 local route to damage-inference [C2]. (3) Inference results go to edgeHub, which forwards them over AMQP 1.0/TLS to IoT Hub [C4]. (4) The ingest API reads the Event Hubs-compatible endpoint in batches of 500 and upserts results to Cosmos DB and images to Blob Storage by idempotency key [C8]. (5) 60-second health messages feed Log Analytics, the dashboard and the ring gates.

Offline flow. When MPLS and LTE are both down [C12], steps 1–3 continue unchanged: inspection never depends on the cloud [C16]. edgeHub persists outbound messages on the buffer volume. On reconnect it drains damage events first (P0), then results (P1), while stale health messages expire after one hour [C17]. Replays are absorbed by keyed upserts [C20]. Figure 2 (§5) quantifies fill and drain.

2.3 Failure modes

Failure mode	Detection	Automatic response	Operator action
Uplink outage [C12]	Heartbeat age; site Offline on dashboard	Store-and-forward, 7-day TTL, priority drain	None < 4 h; inform Site Lead 4–72 h; escalate > 72 h
Module crash or OOM [C13]	edgeAgent restart count; p95 breach	restartPolicy always, MaxRestartCount 5; spool replays queued frames	Dashboard: Restart module; escalate after 2 restarts
Upstream rejection while online [C14]	Buffer > 80 % while site Online	Messages held, not dropped (TTL 7 d)	Escalate; platform adds S2 units or renews certificate
Bad release [C15]	Flagger checks; ring gate KQL	Flagger reverts traffic; workflow deletes release layer	Confirm on dashboard; Revert PR if needed
Power loss during outage	Device reappears with stale heartbeat	Persistent store survives; edgeAgent starts offline (`UseOfflineCheck`)	Power-cycle per site procedure
Camera or trigger fault	Capture success < 99 %; event-rate anomaly	Snapshot retries; health counters	Check camera; Restart frame-capture

3. Edge runtime and IoT Edge manifest (offline configuration)

3.1 IoT Edge 1.6 LTS pin

- **System modules:** `mcr.microsoft.com/azureiotedge-agent:1.6.0` and `azureiotedge-hub:1.6.0` (Appendix D). Exact patch tags are used instead of the floating `:1.6` tag so that an upstream patch cannot bypass the canary; 1.6.x bumps ship as PRs through the same rings.
- **Host packages:** `aziot-edge` and `aziot-identity-service` are apt-pinned to `1.6.*` and held, and the bootstrap edgeAgent image in `/etc/aziot/config.toml` is also `1.6.0` (`host/provision-edge-host.sh`).
- **Guardrail:** the render step in `edge-release.sh` fails any manifest that references an `azureiotedge-*` image older than 1.6. IoT Edge 1.5 LTS (end of support 10 Nov 2026) is never installed.
- **Host OS:** Ubuntu Server 22.04 LTS is Tier 1 for 1.6 on AMD64 (standard support to June 2027); a move to 24.04 (Tier 1 to June 2029) is planned after peak.

3.2 Module definitions

Module	Image (ACR, digest-pinned)	Start	Limits	Key settings
edgeHub	azureiotedge-hub:1.6.0	0	448 MiB	Store on `/srv/edge-buffer/edgehub`; no host ports
frame-capture	frame-capture:1.3.0@sha256	1	192 MiB · 1 CPU	Spool ≤ 1 GB; snapshot timeout 1 s, 3 retries
damage-inference	damage-inference:2.0.0-litert-int8@sha256	2	384 MiB · 2 CPU	LiteRT, 2 threads, confidence 0.45, latency SLO 200 ms
edgeAgent	azureiotedge-agent:1.6.0	—	160 MiB	WaitForAllPulls, MaxRestartCount 5, UseOfflineCheck

Every custom module runs as UID 65532 with a read-only root filesystem, all capabilities dropped and `no-new-privileges`; the `local` log driver caps each container at 3 × 10 MB to protect the eMMC. Tunables (thresholds, retry timings, heartbeat interval) live in module twin desired properties so they can change without an image rollout.

3.3 Offline buffer settings

Setting	Value	Why
`storeAndForwardConfiguration.timeToLiveSecs`	604800 (7 d)	Exceeds the 72 h ceiling with margin; bounds growth at ~2.8 GB [C21]
Route `damageToCloud`	priority 0, TTL 7 d	Damage evidence leaves first after reconnect [C17]
Route `resultsToCloud`	priority 1, TTL 7 d	All results kept through the ceiling
Routes `*HealthToCloud`	priority 2, TTL 1 h	Stale health is worthless and must not compete for bandwidth
Route `captureToInference` (local)	priority 0, TTL 12 h	Durable handoff during inference restarts [C2]
edgeHub `StorageFolder` + bind mount	`/srv/edge-buffer/edgehub:/iotedge/storage`	Persistent queue on the dedicated volume [C5]
`UsePersistentStorage` / `OptimizeForPerformance`	true / false	Write before forwarding; lower RocksDB memory on a 2 GB device
`RocksDB_MaxTotalWalSize`	128 MiB	Caps write-ahead log on eMMC
`MaxUpstreamBatchSize` / `UpstreamFanOutFactor`	20 / 10	Faster drain (~1 MB per upstream batch) [C19]
`ConnectivityCheckFrequencySecs`	60	Detects reconnection within a minute (default 300)
`UpstreamProtocol`	Amqp	AMQP 1.0/TLS multiplexes all module identities on one connection
edgeAgent `ModuleUpdateMode` / `UseOfflineCheck`	WaitForAllPulls / true	No module stops until every new image is local; agent starts while offline [C6]

4. Container image and Helm chart specification

4.1 Edge-tuned Dockerfile (Appendix C)

- **Base images:** `mcr.microsoft.com/azurelinux/base/core:3.0` for the build stage and `azurelinux/distroless/python:3.12` for runtime. The final image has no shell or package manager, which shrinks the CVE surface on 50 unattended sites. Azure Linux 3.0 ships Python 3.12, so build and runtime minors match.

- **Minimal layers:** dependencies are installed first from a hash-pinned `requirements.txt` (`ai-edge-litert==2.3.0`, numpy, pillow, azure-iot-device 2.x) as wheels only, so the layer caches between model updates. The model is copied and checked against the `MODEL_SHA256` recorded in the release PR. Only `deps/`, `model/` and `src/` reach the final stage.
- **Runtime:** `MODEL_RUNTIME=litert`, `LITERT_NUM_THREADS=2`, non-root `USER 65532`. The model is baked into the image so that image digest = model version, and rollback of either is a single operation.

Resource hint (LABEL)	damage-inference	frame-capture	Enforced by
CPU request / limit	1 / 2 cores	0.5 / 1 core	Limit: `NanoCpus`; request: `CpuShares` weight (1024 / 512)
Memory request / limit	256 / 384 MiB	128 / 192 MiB	`MemoryReservation` (request); `Memory` = `MemorySwap` (limit, no swap)

Images are pushed to ACR Premium, geo-replicated to the regions serving the 50 sites, and referenced by tag **and** digest so a re-pushed tag cannot change what devices run.

4.2 Helm chart specification (Appendix B)

`edge-inference-orchestrator` (chart 0.2.0, app 1.0.0) deploys only the central control plane. Key values and the changes made to the template:

Value	Setting	Rationale
`edge.enabled`	false (cloud profile)	**Edge vs cloud toggle.** `true` + `values-edge-profile.yaml` gives a single-replica, no-mesh profile for an edge-adjacent AKS Edge Essentials cluster; never used for the 2 GB IoT Edge devices.
`replicaCount` / resources	3 · 250m/512Mi → 1/1Gi	~4.6 events/s average fleet ingest; >10× headroom for post-outage drain
`deploymentStrategy`	Canary (Flagger, `gatewayapi:v1`)	10 % steps to 50 %, 1-min interval, 3 failed checks → rollback [C9][C24]
`ingress` → `gatewayApi`	Ingress disabled; Gateway API (Istio add-on)	AKS managed NGINX ingress is security-patch-only through Nov 2026 — same deprecation trap as IoT Edge 1.5
`lifecycleHooks`	pre: db-migrate · post: smoke test	Idempotent Cosmos provisioning; end-to-end synthetic event incl. duplicate rejection
`gitops`	flux-v2-aks-extension, branch `release`, 5 min	No hand-applied manifests [C23]
`telemetry.syncProtocol`	amqp (template default mqtt)	Matches edgeHub upstream; MQTT/AMQP over TLS only, never gRPC/plain HTTP/2
`telemetry` batch / dedupe	500 · `idempotency-key` · 14 days	Dedupe window > 2× device TTL [C8]
`concurrencyLimits.maxInFlightSyncBatches`	4 per replica (12 total)	Caps Cosmos write pressure after a mass reconnect
PDB / topology spread	minAvailable 2 · zone skew 1	Zero-downtime node maintenance

5. Communication and resilience design

5.1 Communication patterns

Hop	Pattern	Transport	Guarantee
PLC / camera → frame-capture	Event trigger + request/response snapshot	Site LAN (not cloud sync)	Retried; failures counted in health

Hop	Pattern	Transport	Guarantee
frame-capture → damage-inference	Async message + file spool	edgeHub local route (MQTT/TLS on device)	At-least-once, persisted [C2]
damage-inference → IoT Hub	Async telemetry, store-and-forward	AMQP 1.0/TLS 5671	At-least-once, priority + TTL [C4]
IoT Hub → ingest API	Checkpointed stream consumer	Event Hubs-compatible endpoint	At-least-once → effectively-once by key [C20]
Cloud → device	Desired properties, direct methods	Same AMQP connection	Twin converges after reconnect

5.2 Retry, backoff and batching parameters

Hop	Max retries	Initial	Multiplier / cap	Jitter	When exhausted
Camera snapshot	3	100 ms	×2 / 400 ms	±20 %	Log `capture_failed`, health counter
Module → edgeHub (local)	Unlimited	1 s	×2 / 60 s	±25 %	Never drop; local broker returns quickly
edgeHub → IoT Hub	Built-in	—	connectivity check 60 s; op timeout 20 s	—	Held until TTL (7 d); 80 % alarm first
Ingest → Cosmos / Blob	6	0.5 s	×2 / 30 s	Full	Dead-letter to poison queue; breaker opens after 5 failures for 60 s
Rollout `az` calls	3	10 s	×2 / 60 s	—	Fail the workflow; never a partial ring

Batching. Devices send one message per event (≈ 50 KB, under IoT Hub's 256 KB limit); edgeHub groups up to 20 per upstream batch with fan-out 10 [C19]. The ingest API pulls 500 events per batch, holds at most 4 batches in flight per replica and checkpoints only after the batch is committed, so a crash replays rather than loses data.

5.3 Idempotency

frame-capture generates `idempotency-key = {siteId}-{cameraId}-{captureEpochMs}-{seq}` (site from `IOTEDGE_DEVICEID`, `seq` a per-camera counter persisted on the spool volume). The key is deterministic, so a frame re-processed after a crash or a message re-sent after reconnect carries the same key. It travels as the `idempotency-key` property and `messageId`; the ingest API uses it as the Cosmos DB document id (upsert) and the Blob name (`If-None-Match: *`), within a 14-day dedupe window [C8][C20]. A device-reported daily count per camera is reconciled with accepted counts; a mismatch flags key collisions (for example after a clock reset), which would otherwise drop events silently.

5.4 Sizing and drain

Fleet volume is 400,000 events/day. At IoT Hub's 4 KB metering each 50 KB event counts as 13 messages, so with health heartbeats the fleet needs ≈ 5.3 M metered messages/day. **S2 × 2 units** provide 12 M/day (44 % used) and 240 sends/s [C7]. Per site, a 6 Mbps drain moves ≈ 15 messages/s, so even 10 sites reconnecting together (≈ 150/s) stay under the throttle. A fleet-wide 72 h outage would exceed one day's quota on replay; the 7-day TTL lets it drain over two days, and the platform team can add S2 units without downtime.

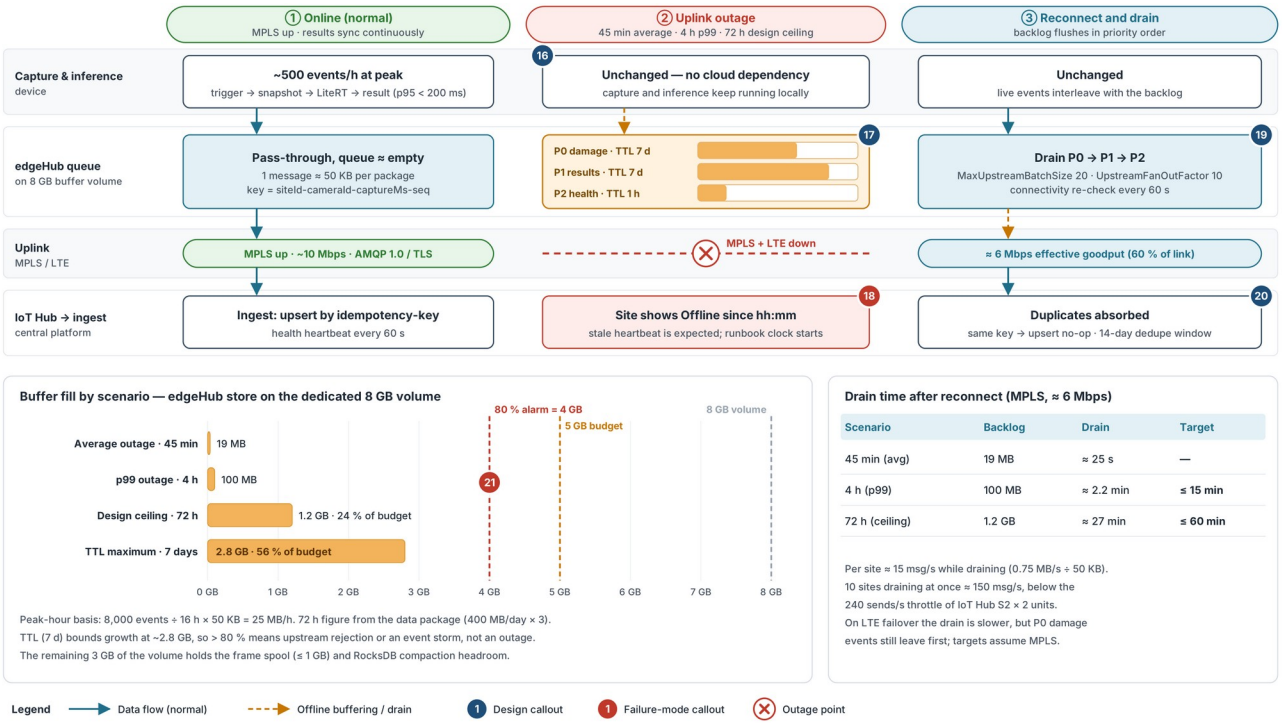


Figure 2. *During an uplink outage the site keeps inspecting packages and queues results by priority, so even the 72-hour ceiling fills only 24 % of the 5 GB buffer budget and drains in about 27 minutes once MPLS returns.*

6. Deployment strategy: canary with GitOps

6.1 Decision: Canary over Blue-Green

Criterion	Canary (chosen)	Blue-Green
Fits 2 GB edge PCs	Yes — one inference stack per device	No — needs a second inference stack (~384 MiB) beside the first
Blast radius of a bad model	1 site, then 5, 20, 50 [C25]	Whole environment at cutover
Detects model regression	Real traffic per ring; damage-rate drift vs 7-day baseline	Mostly synthetic checks before cutover
Rollback	Automatic: Flagger reverts traffic; delete layer → base [C26]	Fast switch back, but standby capacity must be kept
Effort / time	≈ 3 days per fleet rollout; gates reuse health metrics	Faster cutover, double infrastructure, manual switch decisions

6.2 Central rollout via Flux v2 (AKS)

1. Engineer opens a PR changing the chart, values or image digest; CI lints and runs unit tests; platform approves [C22].
2. Merge to `release`. The Flux v2 AKS extension (`microsoft.flux`, created by `gitops/flux-config.sh`) reconciles the GitRepository and HelmRelease within 5 minutes [C23].
3. Helm pre-upgrade hook runs the idempotent `db-migrate` job; a failed upgrade is rolled back by HelmRelease remediation.
4. Flagger shifts 10 % → 50 % of traffic in 10 % steps, checking error rate < 1 % and p99 < 500 ms every minute [C24]. Three failed checks return all traffic to the primary automatically.
5. On success Flagger promotes the canary; the post-upgrade smoke test confirms ingest and duplicate rejection end to end.

6.3 Edge ring rollout via az automation

Ring	Sites (cumulative)	Soak	Gate before widening
R0	1 pilot site (1)	24 h	Benchmarks on real traffic; all gate metrics green
R1	+4 (5)	24 h	p95 < 200 ms · errors < 0.5 % · 0 unexpected restarts · sync lag < 10 min · damage drift ≤ 25 %
R2	+15 (20)	12 h	Same gate on R0–R1; platform approval (protected environment)
R3	+30 (50)	24 h watch	Same gate on R0–R2; then promote release to base (priority 10)

A release is a descriptor file `releases/edge/<version>.env` on the `release` branch. Merging it with `RING=ring0` makes the `edge-release` workflow render the layered manifest and create one IoT Hub layered deployment (priority 100 + N) targeting ring 0. Each later PR changes `RING`; the workflow runs the KQL gate on the soaked rings and, if it passes, widens the target condition (`tags.ring IN [...]`). When ring 3 has soaked, the release becomes the new **base** deployment (priority 10) and the layer is removed **[C10][C26]**.

6.4 Zero-downtime mechanics

- **Central:** `maxUnavailable: 0`, PDB `minAvailable: 2`, readiness/startup probes and Flagger traffic shifting; the old primary serves until the canary is promoted.
- **Edge:** `ModuleUpdateMode=WaitForAllPulls` downloads every new image over the 10 Mbps link before any module stops **[C6]**. During the seconds-long container swap, frame-capture keeps running and `frames` messages queue durably in edgeHub, so no package goes uninspected **[C2]**. At 500 events/h the few queued frames clear in about a second.

6.5 Rollback triggers and verification

Trigger	Detected by	Automatic action	Verification
Central error ≥ 1 % or p99 ≥ 500 ms (3 checks)	Flagger	Traffic → primary; canary scaled to 0	Canary panel Failed; primary healthy; §7.1 passes
Helm upgrade or hook failure	Flux HelmRelease	Remediation rollback	HelmRelease Ready on previous revision
Ring gate breach	edge-release workflow (KQL)	Delete release layer → devices revert to base	`runningRelease` metric = 0; base version reported
Device deployment failure	IoT Hub `reportedFailedCount` > 0	Widening blocked	Per-site check via dashboard logs
Operator-observed regression	Health check §7.1	— (manual) PR Revert or Run workflow → rollback	§7.1 green within 20 min, else escalate

Timeline: weeks 1–3 build and prove (int8 benchmark, IaC, pipeline, outage drills); week 4 rings 0–1; week 5 rings 2–3; weeks 6–8 freeze with emergency-only changes on a compressed soak (4 h / 4 h / 2 h) **[C28]**.

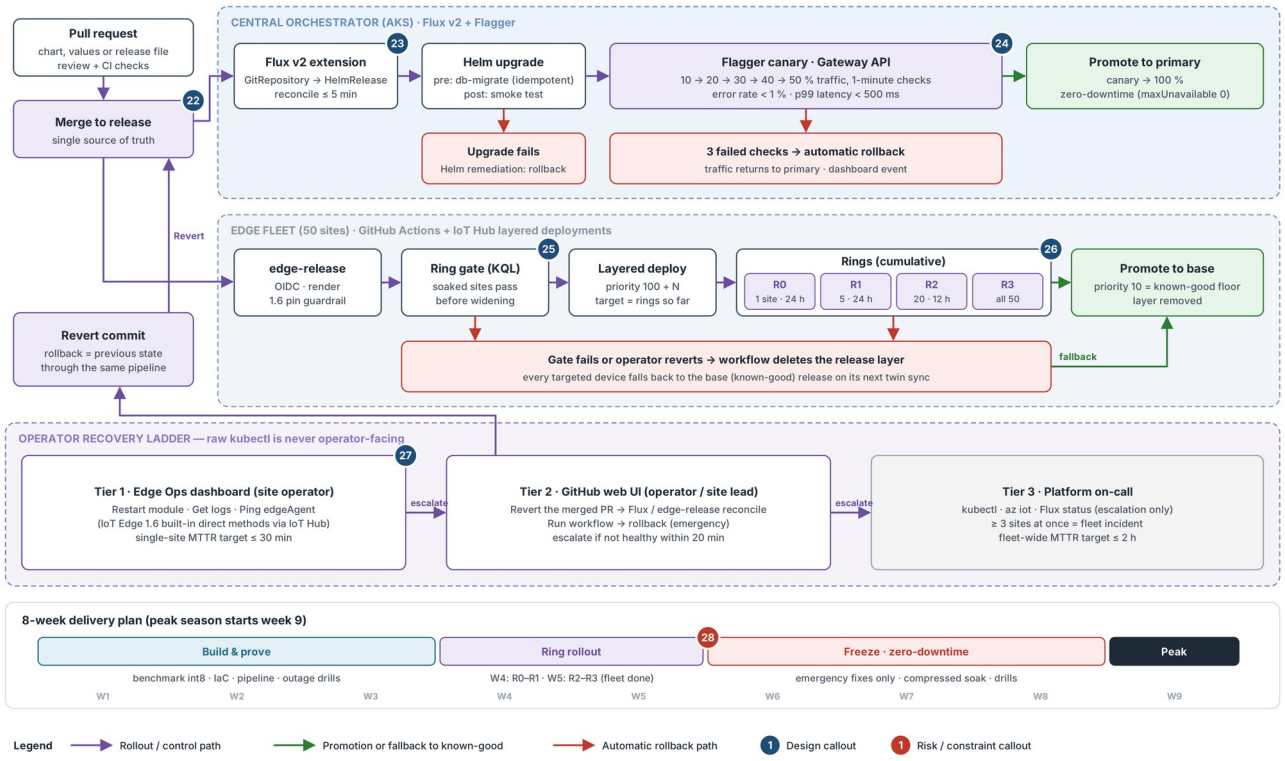


Figure 3. *Every rollout and rollback is a Git commit: Flagger canaries the central orchestrator, IoT Hub layered deployments widen ring by ring behind a metric gate, and operators recover through dashboard actions and PR reverts, never raw kubectl.*

7. Operational runbook

Audience: distribution-center operations staff (no raw kubectl). Operators use the **Edge Ops dashboard** (Entra ID role SiteOperator) and the **GitHub web UI** (Revert and Run workflow buttons). Each dashboard action calls a built-in IoT Edge 1.6 `$edgeAgent` direct method through IoT Hub. Platform-only steps are marked **[PLATFORM]**. A standalone copy is `runbook/RUNBOOK.md`.

SLA / target reference

Metric	Target
Rollout success rate (fleet-wide)	≥ 98 % of targeted devices report the new release within 30 min of being online, without manual action
Sync window after reconnection	≤ 15 min to fully drain a 4 h (p99) backlog; ≤ 60 min for the 72 h ceiling (MPLS)
MTTR — single-site incident	≤ 30 minutes
MTTR — fleet-wide incident	≤ 2 hours
Max concurrent site incidents before escalation	3 sites (≥ 3 at once = fleet incident; page Platform immediately)
Inference latency p95 threshold	< 200 ms

7.1 Health check (run first, every time)

- ☐ Open the dashboard → **Site status**; note Online or “Offline since hh:mm”.
- ☐ Inference latency p95 < 200 ms. If above, go to §7.3.
- ☐ Buffer utilization < 80 % of the 5 GB budget (< 4 GB). If above, go to §7.3 (a normal 72 h outage reaches only ~24 %).

- ☐ Last successful sync < 10 minutes ago (Online). Offline: < 4 h normal; 4–72 h inform Site Lead; > 72 h escalate **[PLATFORM]** [C18].
- ☐ Camera capture success ≥ 99 % and all modules show running on the Modules tile.

7.2 Trigger a rollout

- ☐ Confirm the change is merged to the GitOps `release` branch — Flux reconciles the central orchestrator and the `edge-release` workflow applies edge deployments. Never push manually to the cluster or devices.
- ☐ Edge widening (ring0 → ring3) is a merged PR that changes `RING`; platform approves ring2/ring3 and the gate blocks widening on any failing site.
- ☐ Watch the **Canary** panel (central) and **Ring rollout** panel (targeted / applied / successful / failed).
- ☐ Success: central — Flagger promotes with error < 1 % and p99 < 500 ms on every check; edge — gate metrics in §6.3 green and successful = targeted for online devices.

7.3 Troubleshoot common failures

Symptom	Likely cause	Operator action
Latency p95 above threshold	Resource contention (backlog after restart, log storm)	Dashboard → **Restart inference module**; recheck in 10 min; after 2 restarts escalate
Buffer near capacity (> 80 %)	Upstream rejection while online (quota, certificate) or a trigger storm — not a plain outage [C21]	Online and not draining → escalate **[PLATFORM]**; abnormal event rate → **Restart frame-capture**
Sync not resuming after reconnect	Certificate/auth failure, IoT Hub quota exhausted, route misconfiguration in last release	**Ping edgeAgent**; no reply or lag rising after 15 min → **[PLATFORM]**; right after a release → rollback (§7.4)
Count mismatch / duplicate-reject spike	Idempotency-key collision (e.g. clock reset)	Escalate **[PLATFORM]** for reconciliation
Camera capture failures	Camera power/PoE, cable, credentials	Check camera; **Restart frame-capture**
Rollout stuck in canary analysis	Metric threshold breach	Automatic rollback; confirm on Canary panel, no action needed
Edge ring gate failed	Gate metric breach on soaked sites	Layer removed automatically; confirm sites show the known-good version

7.4 Rollback

- ☐ Identify the last known-good release tag (dashboard **Releases** panel shows each site's base version).
- ☐ Central: open the merged PR → **Revert** → merge; Flux reconciles within 5 min — no kubectl.
- ☐ Edge: **Revert** the release PR (narrows the ring or removes the release); emergency: Actions → `edge-release` → **Run workflow → rollback**.
- ☐ Confirm §7.1 passes and the Ring rollout panel shows affected devices on the known-good version.
- ☐ Not resolved within 20 minutes → escalate **[PLATFORM]**.

7.5 Escalation path

1. Operator → Site Lead (immediately, if the health check fails twice in a row).
2. Site Lead → Platform Team on-call (if §7.3/§7.4 does not resolve within the MTTR target, or ≥ 3 sites are affected) [C27].
3. Platform Team → raw kubectl / cluster-level diagnostics (platform-only, never operator-facing).

7.6 [PLATFORM] Diagnostic commands (not for operator use)

```
HUB=logistics-edge-hub; DEV=dc-017-edge01
# Edge device state (no SSH needed) — runtime must report 1.6.x
az iot hub module-twin show -n $HUB -d $DEV -m '$edgeAgent' \
  --query "properties.reported.{runtime:systemModules.edgeAgent.settings.image, modules:modules}"
az iot hub invoke-module-method -n $HUB -d $DEV -m '$edgeAgent' --mn GetModuleLogs \
```

```
  --mp '{"schemaVersion":"1.0","items":[{"id":"damage-inference","filter":{"tail":200}}],"encoding":"none","contentType":"text"}'
az iot hub invoke-module-method -n $HUB -d $DEV -m '$edgeAgent' --mn RestartModule \
  --mp '{"schemaVersion":"1.0","id":"edgeHub"}'
az iot edge deployment show-metric -n $HUB -d inf-2-0-0-litert-int8 -m reportedFailedCount --metric-type system
az iot hub monitor-events -n $HUB -d $DEV --properties anno sys --timeout 60
# Central AKS (platform only)
kubectl -n edge-inference get canary,deploy,pods
kubectl get gitrepositories,kustomizations,helmreleases -A
az k8s-configuration flux show -g rg-edge-central -c aks-edge-central -t managedClusters -n edge-inference
# Never print device connection strings; devices use X.509 via DPS.
```

8. Design rationale, risks and self-check

8.1 Design rationale

My top design decision is to roll out every edge release as a ring-based canary (1 → 5 → 20 → 50 sites) using layered IoT Hub deployments on Azure IoT Edge 1.6 LTS, with a metric gate and automatic fallback to the last known-good release, instead of a blue-green swap. A bad model or image can then leave damage undetected at one pilot site for at most one soak period instead of at all 50 sites, and operators can roll it back with a PR revert rather than kubectl, which directly limits the financial loss from missed damage. The effort is modest — about three days per fleet rollout, gated on health metrics the system already collects — and unlike blue-green it needs no second inference stack on the 2 GB edge PCs, so all 50 sites can be on the release by week 5, before the week-6 freeze.

8.2 Risks and mitigations

Risk	Impact	Mitigation
int8 accuracy below mAP@0.5 0.88	Missed damage	Week-1 gate on holdout + reference PC; fallback ONNX Runtime CPU (fp32 or QDQ-int8) with memory re-check
Memory pressure on 2 GB PCs	OOM restarts	Hard limits, swap off, `OptimizeForPerformance=false`, R0 soak watches restarts
Fleet-wide outage replay exceeds daily quota	Delayed sync	7-day TTL, temporary S2 scale-out, quota alert at 80 %
Ubuntu 22.04 standard support ends June 2027	Patch gap	Migrate to Ubuntu 24.04 (IoT Edge 1.6 Tier 1) after peak
eMMC wear from queue and spool writes	Device failure	Dedicated volume, `noatime`, log rotation, wear telemetry
Operators unfamiliar with Git	Slow rollback	Dashboard links straight to Revert / Run workflow; rollback drill in week 6

8.3 Self-check against the assignment

Requirement	Where	Status
Architecture overview + diagram: edge components, storage, queue, agent, orchestrator, telemetry, GitOps; normal vs offline; failure modes	§2, Figures 1–2	Done
Helm chart spec + values with edge/cloud toggles	§4.2, Appendix B	Done
Edge Dockerfile: Azure Linux/distroless base, minimal layers, resource hints, requests/limits	§4.1, Appendix C	Done
IoT Edge manifest: module definitions, offline buffer settings, runtime and image tags	§3, Appendix D	Done
IoT Edge pinned to 1.6 LTS; 1.5 LTS not used	§3.1, Appendix D	Done
Communication patterns, retry/backoff, idempotency, batching with example parameters	§5	Done
Canary vs Blue-Green decision; Flux v2 rollout; git revert / az recovery; kubectl platform-only; rollback triggers and verification	§6, Figure 3	Done

Requirement	Where	Status
Runbook: rollouts, health checks, common failures, operator recovery, escalation; commands for platform only	§7	Done
ACR geo-replicated; numbered callouts and one-sentence captions	Figures 1–3, Appendix A	Done
2–3 sentence design rationale	§8.1 and rationale .txt	Done

8.4 Sources checked (October 2026)

- Microsoft Learn — IoT Edge version history and supported platforms (1.6 LTS to 14 Nov 2028; 1.5 LTS to 10 Nov 2026; Tier 1 OS list).
- Azure/iotedge `doc/EnvironmentVariables.md` — edgeAgent and edgeHub settings used in §3.3.
- Microsoft Learn — IoT Hub quotas and throttling (4 KB metering, 256 KB message size, S2 throttle).
- AKS engineering blog (13 Nov 2025) — Application Routing NGINX supported through Nov 2026; Gateway API recommended.
- PyPI `ai-edge-litert` 2.3.0 — x86-64 Linux wheels for CPython 3.10–3.14.

Appendix A. Callout key (Figures 1–3)

#	Finding	Section
1	Event-triggered capture: the PLC package-present signal plus one ONVIF snapshot per package replaces continuous decoding of 4–8 video streams, protecting the 4-core CPU budget.	§2, §3
2	Durable local handoff: JPEG spool on the buffer volume plus an edgeHub local route (P0, TTL 12 h) means no package is missed while a module restarts or updates.	§3, §6
3	Runtime: LiteRT int8 with XNNPACK CPU kernels, 2 threads, 384 MiB memory cap; must pass mAP@0.5 ≥ 0.88 and p95 < 200 ms on reference hardware before Ring 0.	§1, §3
4	edgeHub store-and-forward: damage (P0) and results (P1) keep 7 days, health (P2) 1 hour; upstream protocol pinned to AMQP 1.0 over TLS.	§3, §5
5	Dedicated 8 GB filesystem: 5 GB edgeHub budget (data-package 4× margin), 1 GB frame spool, 2 GB RocksDB/compaction headroom; a full buffer can never fill the OS disk.	§3
6	edgeAgent 1.6.0 pulls every new image before restarting any module (ModuleUpdateMode = WaitForAllPulls) and exposes the built-in direct methods behind dashboard actions.	§3, §7
7	IoT Hub sized at S2 × 2 units: ≈ 5.3 M of 12 M metered 4 KB messages per day (44 %), 240 sends/s; devices use DPS with X.509.	§5
8	Idempotent ingest: key = siteId-cameraId-captureMs-seq is generated at capture, so retries and replays upsert the same record; 14-day dedupe window.	§5
9	Central orchestrator canary uses Flagger with Gateway API on the AKS Istio add-on, avoiding Ingress-NGINX (security-patch-only on AKS through Nov 2026).	§4, §6
10	Edge GitOps: GitHub Actions renders the manifest and widens one layered IoT Hub deployment ring by ring behind a Log Analytics gate.	§6
11	Operator recovery uses dashboard actions (direct methods) and the GitHub Revert button; kubectl is platform-only escalation.	§7
12	Failure mode: MPLS and LTE both down (45 min average, 4 h p99, 72 h ceiling) → site runs offline and buffers (Figure 2).	§5
13	Failure mode: inference crash or OOM → restartPolicy always, MaxRestartCount 5, queued frames replay from spool; operator can restart from dashboard.	§3, §7
14	Failure mode: link up but IoT Hub rejects (quota exhausted, certificate/auth) → buffer grows while online → 80 % alarm and escalation.	§5, §7
15	Failure mode: bad release → automatic rollback (Flagger after 3 failed checks; edge ring gate deletes the release layer).	§6
16	Local autonomy: capture and inference have no cloud dependency, so damage detection never stops during an outage.	§5
17	Priority queues: damage events leave first after reconnect; stale health messages expire after 1 hour instead of competing for bandwidth.	§3, §5
18	Dashboard shows the site Offline with heartbeat age; < 4 h normal, 4–72 h inform Site Lead, > 72 h escalate.	§7
19	Drain tuning: MaxUpstreamBatchSize 20, UpstreamFanOutFactor 10, ConnectivityCheckFrequencySecs 60; targets ≤ 15 min (4 h backlog) and ≤ 60 min (72 h).	§5
20	At-least-once delivery is made effectively-once by keyed upserts in Cosmos DB and If-None-Match writes to Blob Storage.	§5
21	Because the 7-day TTL caps outage growth at ~2.8 GB (56 %), crossing the 80 % alarm signals upstream rejection or an event storm, not an ordinary outage.	§5, §7
22	Git is the single source of truth: a merged PR is a rollout and a Revert is a rollback, for both the cloud and the edge path.	§6

#	Finding	Section
23	Flux v2 AKS extension reconciles the HelmRelease every 5 min; pre/post-upgrade hooks and Helm remediation roll back failed upgrades.	§4, §6
24	Flagger analysis: 10 % steps to 50 %, 1-minute checks, error rate < 1 % and p99 < 500 ms; 3 failed checks trigger rollback.	§6
25	Ring gate (KQL): p95 < 200 ms, errors < 0.5 %, zero unexpected restarts, sync lag < 10 min, damage-rate drift ≤ 25 % vs 7-day baseline.	§6
26	Base deployment (priority 10) is the known-good floor; deleting the release layer returns every targeted device to it.	§6
27	Three-tier recovery ladder (dashboard → GitHub → platform) aligned with MTTR targets of 30 min (site) and 2 h (fleet).	§7
28	Constraint: zero-downtime freeze from week 6, so all 50 sites must be on the release by the end of week 5.	§6

Appendix B. Artifact index

Path (artifact bundle)	Purpose
helm/edge-inference-orchestrator/Chart.yaml, values.yaml	Helm chart spec and values (cloud profile) — listed below
helm/edge-inference-orchestrator/values-edge-profile.yaml	Edge-adjacent overlay (`edge.enabled=true`), not used for this fleet
docker/Dockerfile.edge	Edge inference image — Appendix C
iot-edge-manifest/deployment.template.json	Base deployment (IoT Edge 1.6.0, offline buffer) — Appendix D
iot-edge-manifest/inference.layer.template.json	Release layer used for ring rollouts
scripts/edge-release.sh, scripts/ring-gate.kql	Render + guardrail, ring targeting, rollback; promotion gate
.github/workflows/edge-release.yml, releases/edge/2.0.0.env	Edge GitOps workflow and release descriptor
gitops/flux-config.sh, gitops/clusters/prod/edge-inference/*.yaml	Flux v2 extension bootstrap, HelmRelease, Flagger Canary + metric templates
host/provision-edge-host.sh	Ubuntu 22.04 image: IoT Edge 1.6 pin, buffer volume, DPS X.509
runbook/RUNBOOK.md	Operational runbook (§7)
(not included) src/, requirements.txt, model/	Application code and hash-pinned dependencies owned by the ML team; consumed by Dockerfile.edge

B.1 Chart.yaml

```
apiVersion: v2
name: edge-inference-orchestrator
description: Central AKS orchestrator for the edge inference pipeline (telemetry ingest, device registry sync, dashboard, GitOps-
managed)
type: application
# Chart version: bump on any template/values change (SemVer). Flux HelmRelease pins a range (0.2.x).
version: 0.2.0
# appVersion tracks the orchestrator image tag in values.yaml (image.tag)
appVersion: "1.0.0"
kubeVersion: ">=1.30.0-0"
keywords:
  - edge-inference
  - iot-edge
  - gitops
  - flagger
annotations:
  # Edge devices are NOT deployed by this chart. They run Azure IoT Edge 1.6 LTS
  # (edgeAgent/edgeHub 1.6.0) and receive modules via IoT Hub deployment manifests.
  edge-inference/iot-edge-runtime-baseline: "1.6 LTS (1.6.0)"
  edge-inference/managed-by: "flux-v2-aks-extension"
maintainers:
  - name: Dennis (Platform Engineering)
```

```
    email: platform-team@example.com
```

B.2 values.yaml

```
# values.yaml — central AKS orchestrator
# This chart deploys ONLY the cloud-side control plane (ingest API, device
# registry sync, telemetry pipeline, dashboard). Azure IoT Edge modules are
# NOT deployed via Helm — they are deployed to devices via IoT Hub
# deployment manifests. See iot-edge-manifest/deployment.template.json.
#
# EDGE vs CLOUD TOGGLE
#   edge.enabled=false (default) -> cloud profile: 3 replicas, Gateway API, Flagger canary.
#   edge.enabled=true            -> edge-adjacent profile for AKS Edge Essentials clusters
#                                   (Azure IoT Operations scope) using values-edge-profile.yaml.
#   The 50 distribution centers run classic Azure IoT Edge 1.6 LTS on 2 GB PCs, so
#   edge.enabled stays FALSE for this fleet. Never set it for classic IoT Edge devices.

edge:
  enabled: false        # true only for edge-adjacent AKS Edge Essentials clusters (Azure IoT Operations scope), never for classic IoT
Edge devices
  profile:              # applied by templates only when edge.enabled=true
    replicaCount: 1
    gatewayApi: false   # single-node cluster: ClusterIP + local LB, no mesh
    canary: false       # single replica -> RollingUpdate only
    storageClass: local-path
    resources:
      requests: { cpu: "100m", memory: "256Mi" }
      limits:   { cpu: "500m", memory: "512Mi" }

image:
  registry: logisticsedgeacr.azurecr.io   # ACR Premium, geo-replicated to the regions serving the 50 sites (placeholder name)
  repository: edge-inference/orchestrator
  tag: "1.0.0"                            # immutable tag; Flux image policy pins digest in the release branch
  pullPolicy: IfNotPresent

replicaCount: 3

resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
# Sizing note: fleet ingest averages ~20 GB/day (~0.25 MB/s) and ~4.6 events/s;
# 3 x 250m requests give >10x headroom for post-outage drain bursts.

podDisruptionBudget:
  enabled: true
  minAvailable: 2

topologySpreadConstraints:
  enabled: true
  topologyKey: topology.kubernetes.io/zone
  maxSkew: 1

probes:
  readinessProbe:
    httpGet:
      path: /healthz/ready
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
  livenessProbe:
    httpGet:
      path: /healthz/live
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 15
  startupProbe:
    httpGet:
      path: /healthz/startup
      port: 8080
    failureThreshold: 30
    periodSeconds: 5

deploymentStrategy:
  type: Canary            # DECISION: Canary (Flagger + Flux). Blue-Green rejected: doubles capacity, manual cutover.
  rollingUpdate:          # used by the Flagger-managed primary Deployment and when edge.enabled=true
```

```
    maxSurge: 1
    maxUnavailable: 0
  canary:
    provider: gatewayapi:v1   # Gateway API (HTTPRoute weights) — avoids Ingress-NGINX (security-patch-only on AKS through Nov 2026)
    analysis:
      interval: 1m
      threshold: 3            # failed checks before automatic rollback
      stepWeight: 10          # % traffic shifted per interval
      maxWeight: 50
      metrics:
        requestSuccessRateMin: 99      # %
        requestDurationP99MaxMs: 500
    webhooks:
      preRollout: http://smoke-tester.edge-inference/run      # synthetic ingest + idempotency check
      rollback: http://dashboard.edge-inference/api/rollout-events

lifecycleHooks:
  helm:
    preUpgrade: "db-migrate job — idempotent Cosmos DB container/index provisioning (results by idempotency-key, partition /siteId)"
    postUpgrade: "smoke-test job — /healthz/ready on all pods + one synthetic event end-to-end, asserts duplicate is rejected"

gitops:
  managedBy: flux-v2-aks-extension   # Do not hand-apply with kubectl in production; reconciled via Flux GitRepository + HelmRelease
  sourceRepo: "https://github.com/logistics-platform/edge-inference-gitops.git"   # placeholder org/repo
  branch: release
  path: ./clusters/prod/edge-inference
  interval: 5m
  targetNamespace: edge-inference

ingress:
  enabled: false          # Replaced by gatewayApi below (AKS app-routing NGINX is security-patch-only through Nov 2026)
  className: ""
  tls:
    enabled: true

gatewayApi:
  enabled: true
  gatewayClassName: istio          # AKS Istio-based service mesh add-on (Gateway API)
  hostname: edge-ops.logistics.example.com
  tls:
    enabled: true
    certificateSecret: edge-ops-tls  # synced from Key Vault via Secrets Store CSI driver

identity:
  workloadIdentity: true            # Entra Workload ID; no secrets in values
  clientId: "00000000-0000-0000-0000-000000000000"   # placeholder

iotHub:
  hostname: logistics-edge-hub.azure-devices.net    # placeholder; S2 x 2 units (see sizing)
  consumerGroup: orchestrator-ingest

telemetry:
  syncProtocol: amqp         # MQTT v3.1.1/v5 or AMQP 1.0 over TLS — Azure IoT Hub-supported transports only. Do not use gRPC/plain
HTTP2 for device sync.
                             # AMQP chosen: edgeHub multiplexes all module identities over one upstream connection
(UpstreamProtocol=Amqp on devices).
  batchSize: 500             # events per checkpointed batch from the IoT Hub Event Hubs-compatible endpoint
  dedupeKey: idempotency-key
  dedupeWindowDays: 14       # > 2x the 7-day device TTL so late replays are still deduplicated
  retry:
    maxRetries: 6
    initialBackoffMs: 500
    backoffMultiplier: 2.0
    maxBackoffMs: 30000
    jitter: full
  circuitBreaker:
    consecutiveFailures: 5
    openSeconds: 60
  deadLetter:
    enabled: true
    target: sb-queue-ingest-poison

concurrencyLimits:
  maxInFlightSyncBatches: 4  # per replica (12 fleet-wide with 3 replicas); caps write pressure on Cosmos DB after mass reconnect
```

Appendix C. Dockerfile.edge

```
# syntax=docker/dockerfile:1.7
# Dockerfile — edge inference module (damage-inference)
# Base: Azure Linux 3.0 build stage + Azure Linux distroless Python 3.12 runtime,
# per Microsoft container base image guidance. No shell, no package manager in the
# final image -> smaller CVE surface on 50 unattended sites.
# Host OS is Ubuntu Server 22.04 + IoT Edge 1.6 LTS; this image is host-OS agnostic.
#
# Runtime decision: LiteRT (formerly TensorFlow Lite), int8-quantized YOLO detector,
# XNNPACK CPU kernels. Chosen because sites are CPU-only 4-core x86_64 with 2 GB RAM:
# int8 runs natively on CPU and the 7 MB model leaves memory headroom for edgeHub,
# whereas the 25 MB fp16 ONNX export has no fp16 hardware path on these CPUs.

ARG PYTHON_MINOR=3.12

########## build stage — nothing here ships ##########
FROM mcr.microsoft.com/azurelinux/base/core:3.0 AS build
ARG MODEL_SHA256
# Build-time deps only. Azure Linux 3.0 ships Python 3.12 = same minor as the runtime image.
RUN tdnf install -y python3 python3-pip ca-certificates && tdnf clean all
WORKDIR /build

# 1) dependencies first (cache-friendly layer); wheels only, hash-pinned
COPY requirements.txt .
#    requirements.txt pins: ai-edge-litert==2.3.0, numpy, pillow, azure-iot-device (2.x) — each with --hash
RUN python3 -m pip install --no-cache-dir --only-binary=:all: --require-hashes \
        -r requirements.txt --target=/build/deps \
 && find /build/deps -name "__pycache__" -prune -exec rm -rf {} +

# 2) model artifact — integrity-checked against the hash recorded in the release PR
COPY model/damage-detector-int8.tflite model/
RUN echo "${MODEL_SHA256}  model/damage-detector-int8.tflite" | sha256sum -c -

# 3) application source
COPY src/ src/

########## runtime stage ##########
FROM mcr.microsoft.com/azurelinux/distroless/python:3.12 AS final

WORKDIR /app
COPY --from=build /build/deps  /app/deps
COPY --from=build /build/model /app/model
COPY --from=build /build/src   /app/src

# Model runtime: LiteRT int8 (decision recorded in design rationale)
ENV MODEL_RUNTIME=litert \
    MODEL_PATH=/app/model/damage-detector-int8.tflite \
    MODEL_VERSION=2.0.0-int8 \
    LITERT_NUM_THREADS=2 \
    SPOOL_DIR=/spool \
    PYTHONPATH=/app/deps:/app/src \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Resource hints (declare, do not enforce here — enforced via IoT Edge createOptions HostConfig)
LABEL resource.cpu.request="1" \
      resource.cpu.limit="2" \
      resource.memory.request="256Mi" \
      resource.memory.limit="384Mi" \
      org.opencontainers.image.title="damage-inference" \
      org.opencontainers.image.version="2.0.0" \
      org.opencontainers.image.description="Package damage detection, LiteRT int8 + XNNPACK, IoT Edge 1.6 module"

# Non-root (distroless 'nonroot'); /spool is bind-mounted from the 8 GB buffer volume
USER 65532:65532
ENTRYPOINT ["python3", "-m", "damage_inference"]
```

Appendix D. IoT Edge deployment manifest (deployment.template.json)

`${...}` placeholders are substituted by `scripts/edge-release.sh`, which also stringifies `createOptions` and rejects any runtime image older than 1.6.

```
{
  "$schema-template": "4.0.0",
  "modulesContent": {
```

```
"$edgeAgent": {
  "properties.desired": {
    "schemaVersion": "1.1",
    "runtime": {
      "type": "docker",
      "settings": {
        "minDockerVersion": "v1.41",
        "loggingOptions": "",
        "registryCredentials": {
          "acr": {
            "username": "${ACR_PULL_TOKEN_NAME}",
            "password": "${ACR_PULL_TOKEN_PASSWORD}",
            "address": "${ACR_LOGIN_SERVER}"
          }
        }
      }
    },
    "systemModules": {
      "edgeAgent": {
        "type": "docker",
        "settings": {
          "image": "mcr.microsoft.com/azureiotedge-agent:1.6.0",
          "createOptions": {
            "HostConfig": {
              "Binds": ["/var/lib/edge-agent:/iotedge/storage"],
              "Memory": 167772160,
              "MemorySwap": 167772160,
              "LogConfig": { "Type": "local", "Config": { "max-size": "10m", "max-file": "3" } }
            }
          }
        },
        "env": {
          "StorageFolder": { "value": "/iotedge/storage/" },
          "UsePersistentStorage": { "value": "true" },
          "UpstreamProtocol": { "value": "Amqp" },
          "ModuleUpdateMode": { "value": "WaitForAllPulls" },
          "MaxRestartCount": { "value": "5" },
          "CoolOffTimeUnitInSeconds": { "value": "10" },
          "IntensiveCareTimeInMinutes": { "value": "10" },
          "UseOfflineCheck": { "value": "true" }
        }
      },
      "edgeHub": {
        "type": "docker",
        "status": "running",
        "restartPolicy": "always",
        "startupOrder": 0,
        "settings": {
          "image": "mcr.microsoft.com/azureiotedge-hub:1.6.0",
          "createOptions": {
            "HostConfig": {
              "Binds": ["/srv/edge-buffer/edgehub:/iotedge/storage"],
              "Memory": 469762048,
              "MemorySwap": 469762048,
              "LogConfig": { "Type": "local", "Config": { "max-size": "10m", "max-file": "3" } }
            }
          }
        },
        "env": {
          "StorageFolder": { "value": "/iotedge/storage/" },
          "UsePersistentStorage": { "value": "true" },
          "OptimizeForPerformance": { "value": "false" },
          "UpstreamProtocol": { "value": "Amqp" },
          "MaxUpstreamBatchSize": { "value": "20" },
          "UpstreamFanOutFactor": { "value": "10" },
          "ConnectivityCheckFrequencySecs": { "value": "60" },
          "CloudOperationTimeoutSecs": { "value": "20" },
          "MessageCleanupIntervalSecs": { "value": "1800" },
          "RocksDB_MaxTotalWalSize": { "value": "134217728" },
          "SslProtocols": { "value": "tls1.2,tls1.3" }
        }
      }
    },
    "modules": {
      "frame-capture": {
        "version": "1.0",
        "type": "docker",
        "status": "running",
        "restartPolicy": "always",
        "startupOrder": 1,
```

```
          "settings": {
            "image": "${ACR_LOGIN_SERVER}/edge-inference/frame-capture:1.3.0@${FRAME_CAPTURE_DIGEST}",
            "createOptions": {
              "User": "65532:65532",
              "HostConfig": {
                "Binds": [
                  "/srv/edge-buffer/spool:/spool",
                  "/etc/edge-inference/cameras.json:/config/cameras.json:ro"
                ],
                "Memory": 201326592,
                "MemorySwap": 201326592,
                "MemoryReservation": 134217728,
                "NanoCpus": 1000000000,
                "CpuShares": 512,
                "ReadonlyRootfs": true,
                "Tmpfs": { "/tmp": "rw,size=16m" },
                "CapDrop": ["ALL"],
                "SecurityOpt": ["no-new-privileges"],
                "LogConfig": { "Type": "local", "Config": { "max-size": "10m", "max-file": "3" } }
              }
            }
          },
          "env": {
            "SPOOL_DIR": { "value": "/spool" },
            "CAMERA_CONFIG": { "value": "/config/cameras.json" }
          }
        },
        "damage-inference": {
          "version": "2.0",
          "type": "docker",
          "status": "running",
          "restartPolicy": "always",
          "startupOrder": 2,
          "settings": {
            "image": "${ACR_LOGIN_SERVER}/edge-inference/damage-inference:2.0.0-litert-int8@${INFERENCE_DIGEST}",
            "createOptions": {
              "User": "65532:65532",
              "HostConfig": {
                "Binds": ["/srv/edge-buffer/spool:/spool"],
                "Memory": 402653184,
                "MemorySwap": 402653184,
                "MemoryReservation": 268435456,
                "NanoCpus": 2000000000,
                "CpuShares": 1024,
                "ReadonlyRootfs": true,
                "Tmpfs": { "/tmp": "rw,size=16m" },
                "CapDrop": ["ALL"],
                "SecurityOpt": ["no-new-privileges"],
                "LogConfig": { "Type": "local", "Config": { "max-size": "10m", "max-file": "3" } }
              }
            }
          },
          "env": {
            "MODEL_RUNTIME": { "value": "litert" },
            "LITERT_NUM_THREADS": { "value": "2" },
            "SPOOL_DIR": { "value": "/spool" }
          }
        }
      }
    },
    "$edgeHub": {
      "properties.desired": {
        "schemaVersion": "1.2",
        "routes": {
          "captureToInference": {
            "route": "FROM /messages/modules/frame-capture/outputs/frames INTO
BrokeredEndpoint(\"/modules/damage-inference/inputs/frames\")",
            "priority": 0,
            "timeToLiveSecs": 43200
          },
          "damageToCloud": {
            "route": "FROM /messages/modules/damage-inference/outputs/damage INTO $upstream",
            "priority": 0,
            "timeToLiveSecs": 604800
          },
          "resultsToCloud": {
            "route": "FROM /messages/modules/damage-inference/outputs/results INTO $upstream",
            "priority": 1,
            "timeToLiveSecs": 604800
```

```
        },
        "inferenceHealthToCloud": {
          "route": "FROM /messages/modules/damage-inference/outputs/health INTO $upstream",
          "priority": 2,
          "timeToLiveSecs": 3600
        },
        "captureHealthToCloud": {
          "route": "FROM /messages/modules/frame-capture/outputs/health INTO $upstream",
          "priority": 2,
          "timeToLiveSecs": 3600
        }
      },
      "storeAndForwardConfiguration": {
        "timeToLiveSecs": 604800
      }
    }
  },
  "frame-capture": {
    "properties.desired": {
      "triggerMode": "plc-package-present",
      "snapshotTimeoutMs": 1000,
      "snapshotRetries": 3,
      "snapshotBackoffMs": 100,
      "spoolMaxBytes": 1073741824,
      "heartbeatSecs": 60
    }
  },
  "damage-inference": {
    "properties.desired": {
      "confidenceThreshold": 0.45,
      "damageClasses": ["crushed", "torn", "punctured", "wet"],
      "latencySloMs": 200,
      "heartbeatSecs": 60,
      "localRetry": {
        "initialBackoffMs": 1000,
        "multiplier": 2.0,
        "maxBackoffMs": 60000,
        "jitterPct": 25
      }
    }
  }
 }
}
```